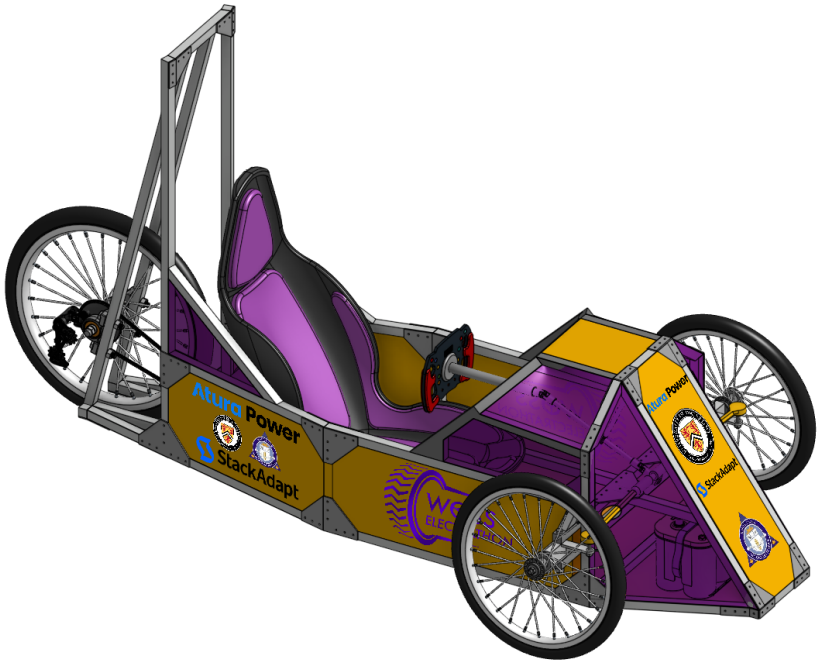
Making a car, then avoiding driving it.

Zane Beeai



June 12, 2025

Outline

Our Ethos

- Minimalist Engineering: simplicity, manufacturability, robustness.
- Quantitative modeling: FEA, drivetrain & electrical analysis.
- Subsystem-based design philosophy.

Design Objectives

- Simplicity Across Subsystems
- DFMA for Manual Tools
- Subsystem Modularity
- Numerical Justification via Simulation
- Electrical Redundancy (telemetry, safeguards)

Material Selection

Aluminum offered ~74% mass reduction compared to steel while maintaining safety margins validated by FEA.

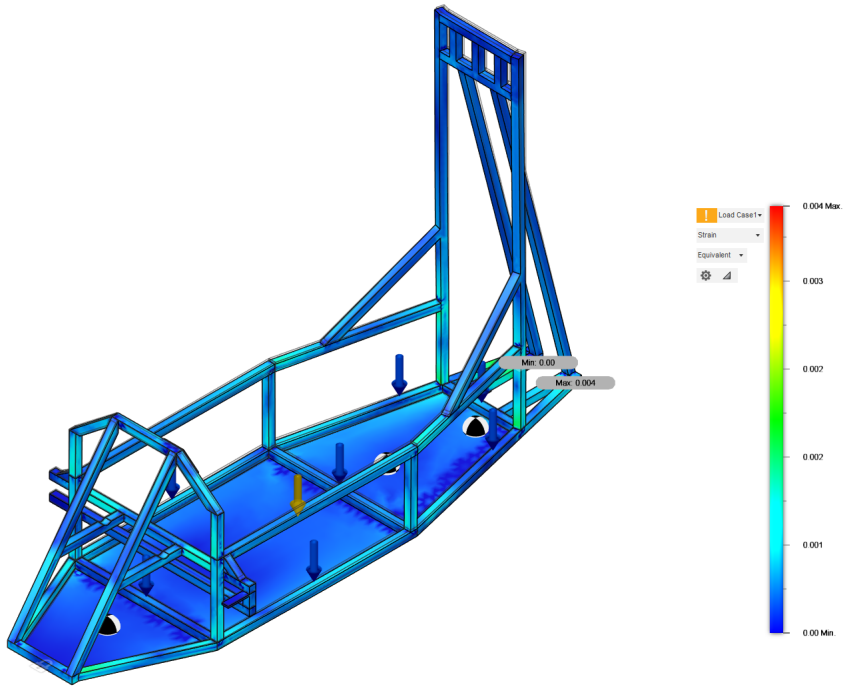


Figure: Figure 1: Frame FEA under 10,000 N load.

Hybrid Assembly

Structural gussets combined with bolts, rivets, and TIG welding reinforced load paths at critical junctions.

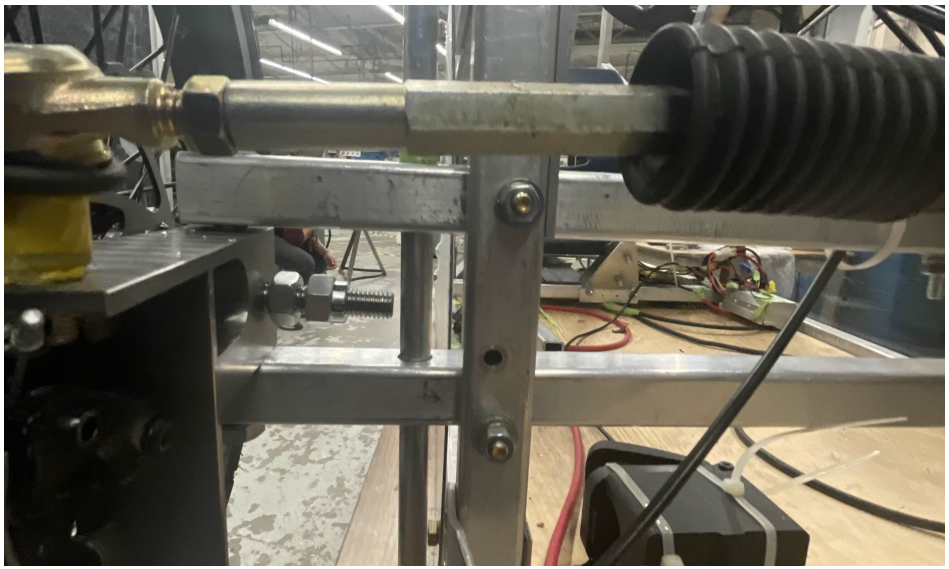


Figure: Figure 2: Modular clearance mount design for track conditions.

Load Distribution

Weight placement balanced the longitudinal COM; battery front-loaded to counter motor mass.

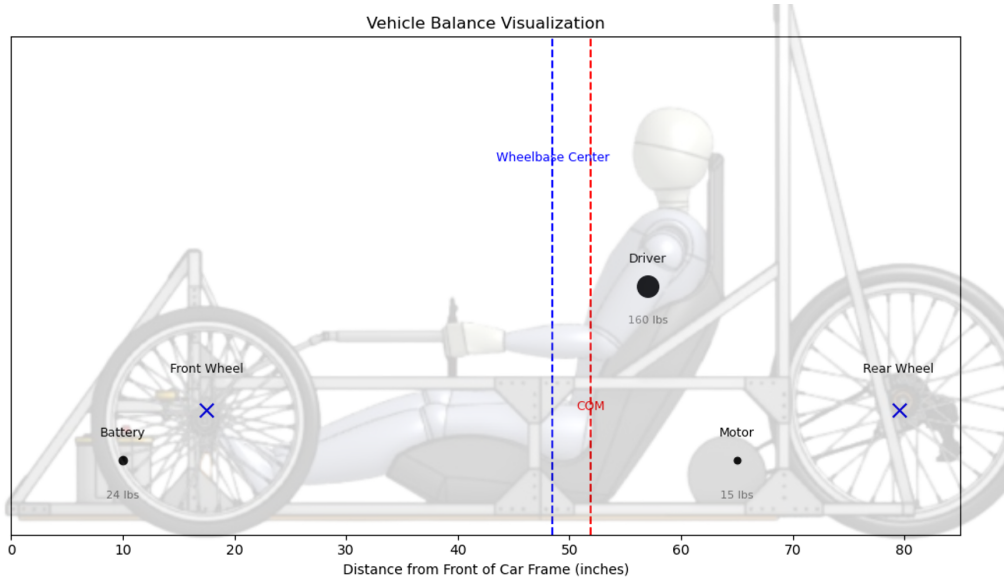


Figure: Figure 3: Longitudinal mass distribution.

FEA Validation

Final frame FEA validated design stability at peak race loads.

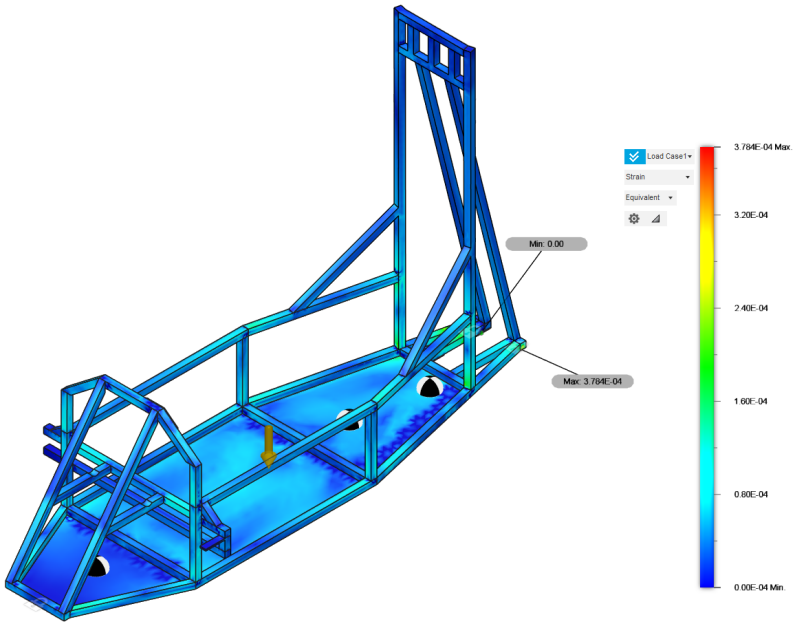


Figure: Figure 4: Von Mises stress simulation.

Stress-Strain Analysis

Strain levels remain far within aluminum elastic region.

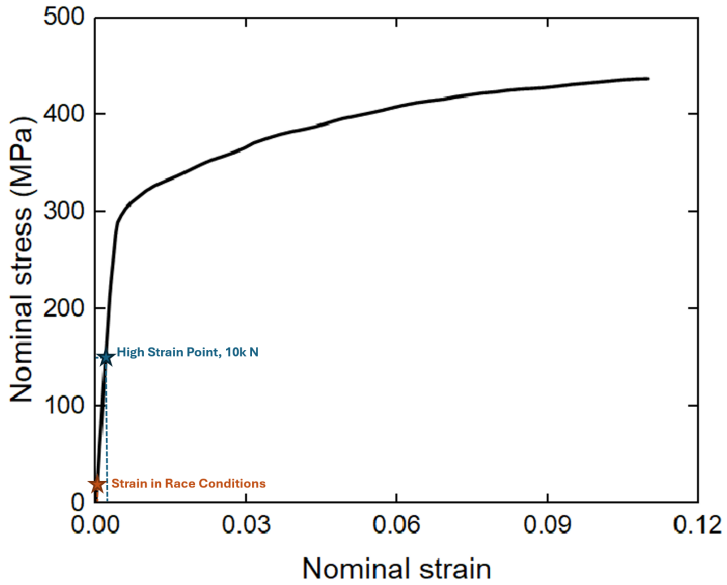


Figure: Figure 5: 1060 aluminum stress-strain curve with FEA data.

Spindle Design

Aluminum spindle failed under braking; steel redesign eliminated excessive deflection.

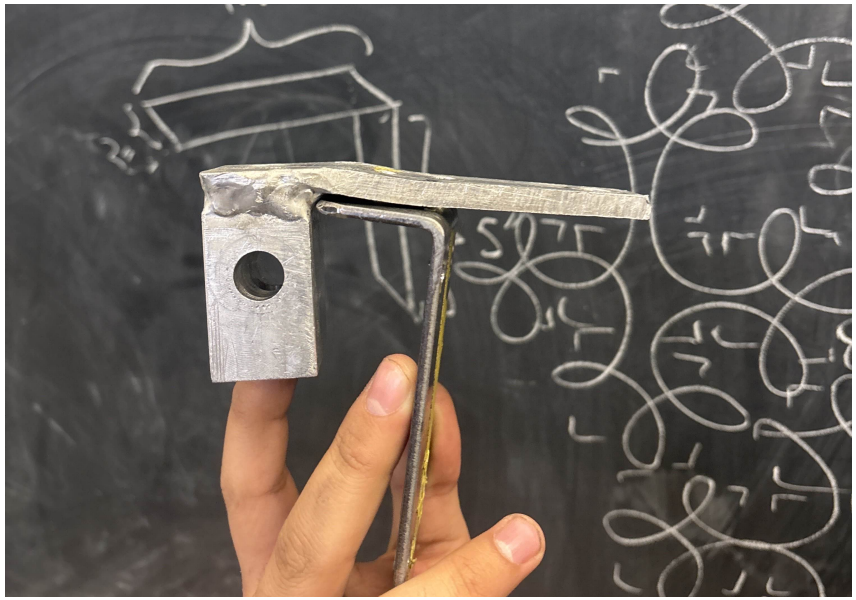


Figure: Figure 6: Aluminum spindle post-failure.

Steel Spindle FEA

1045 steel spindle blocks showed minimal strain under braking simulation.

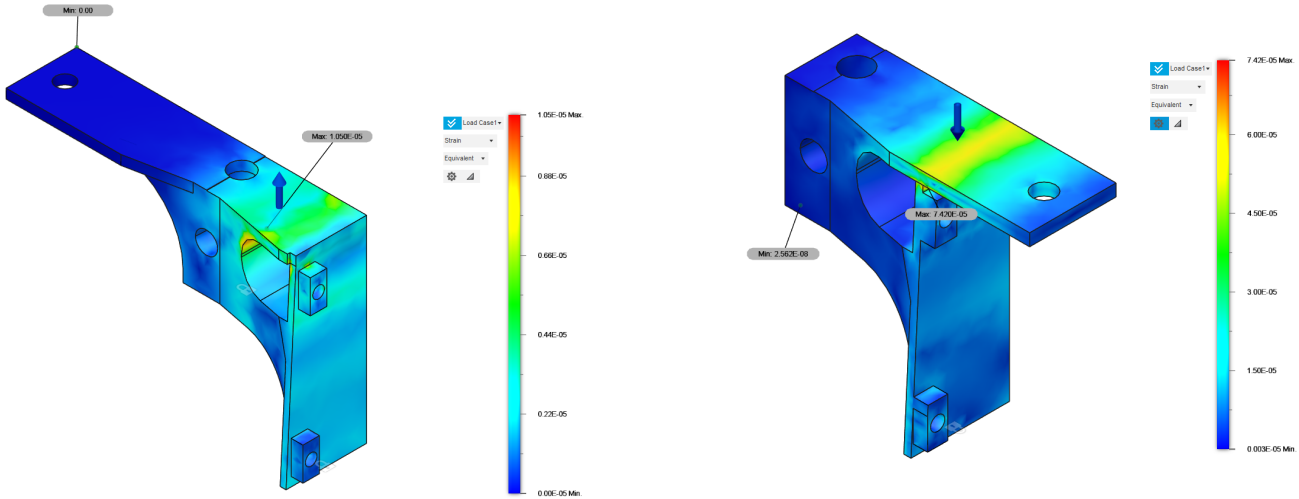


Figure: Figure 7: FEA strain on steel spindles (left/right).

Camber Analysis

Lateral grip maximized near -3° camber (Pacejka-based model).

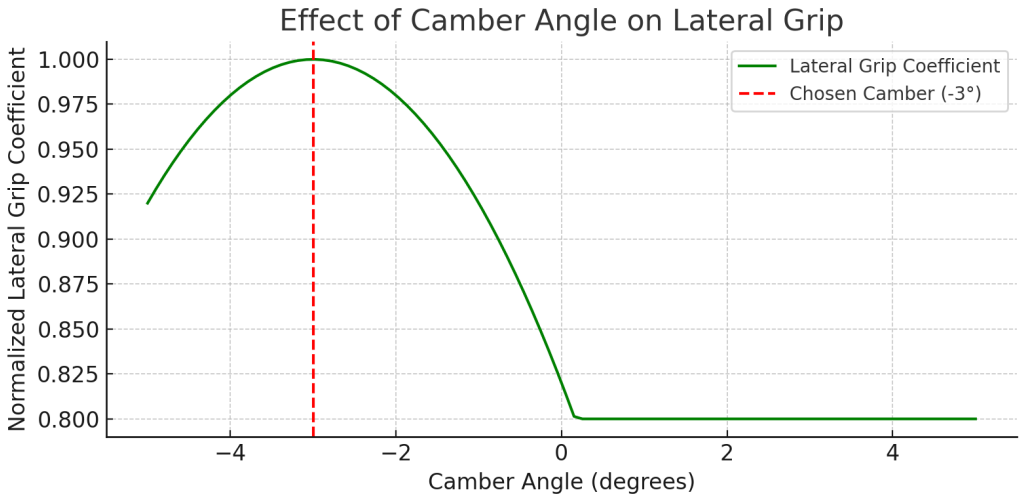


Figure: Figure 8: Camber vs lateral force.

Camber Testing

Visual test confirmed -3° camber in real-world driving.

Figure: Figure 9: Camber confirmation.

Steering Geometry

Ackermann geometry validated for 2.5m turn radius.

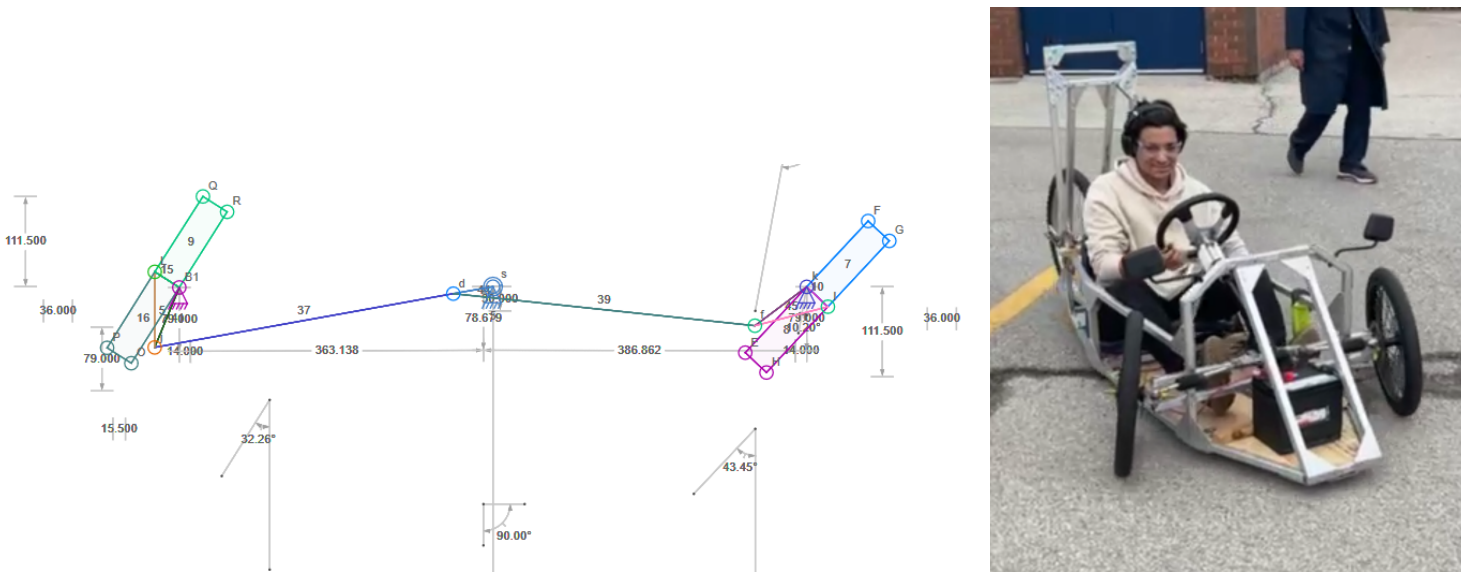


Figure: Figure 10: CAD projection (left), drive test (right).

Motor Selection

AmpFlow E30-400-24 provided optimal efficiency at 480W input.

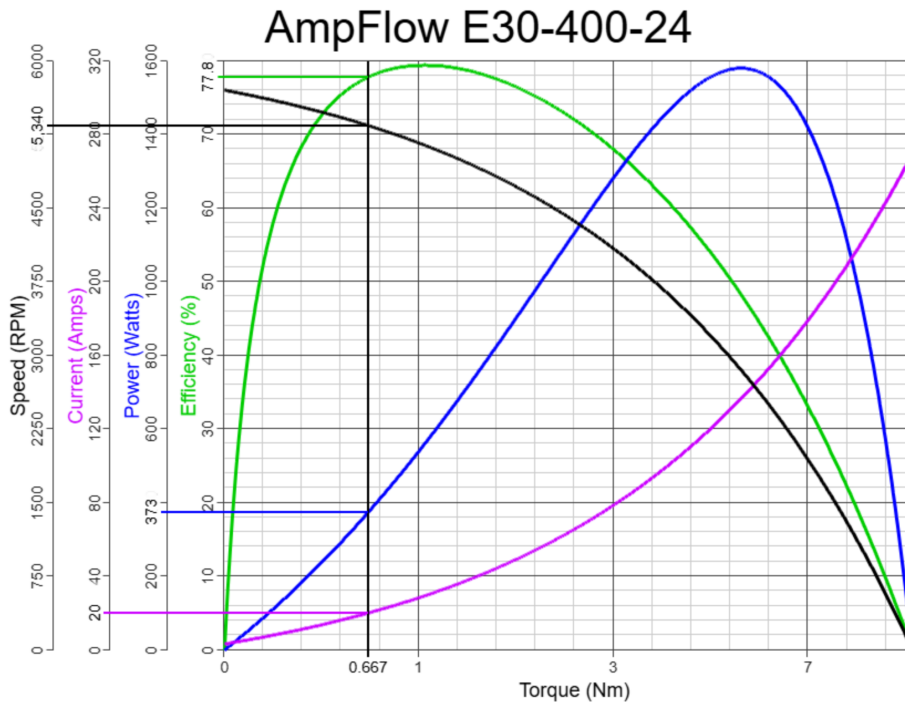


Figure: Figure 11: AmpFlow motor efficiency curve.

Custom Gearbox

Custom 81:16 reduction gearbox minimized sprocket friction losses.

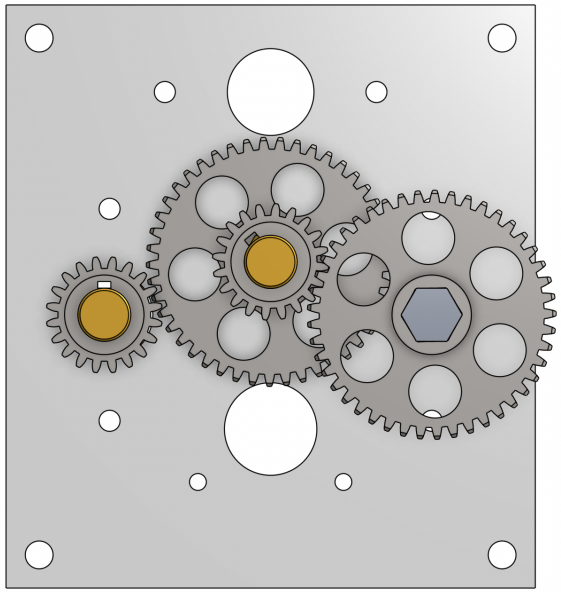


Figure: Figure 12: Custom gearbox design.

Chain Stability

Idler-tensioner reduced chain oscillations and derailments.

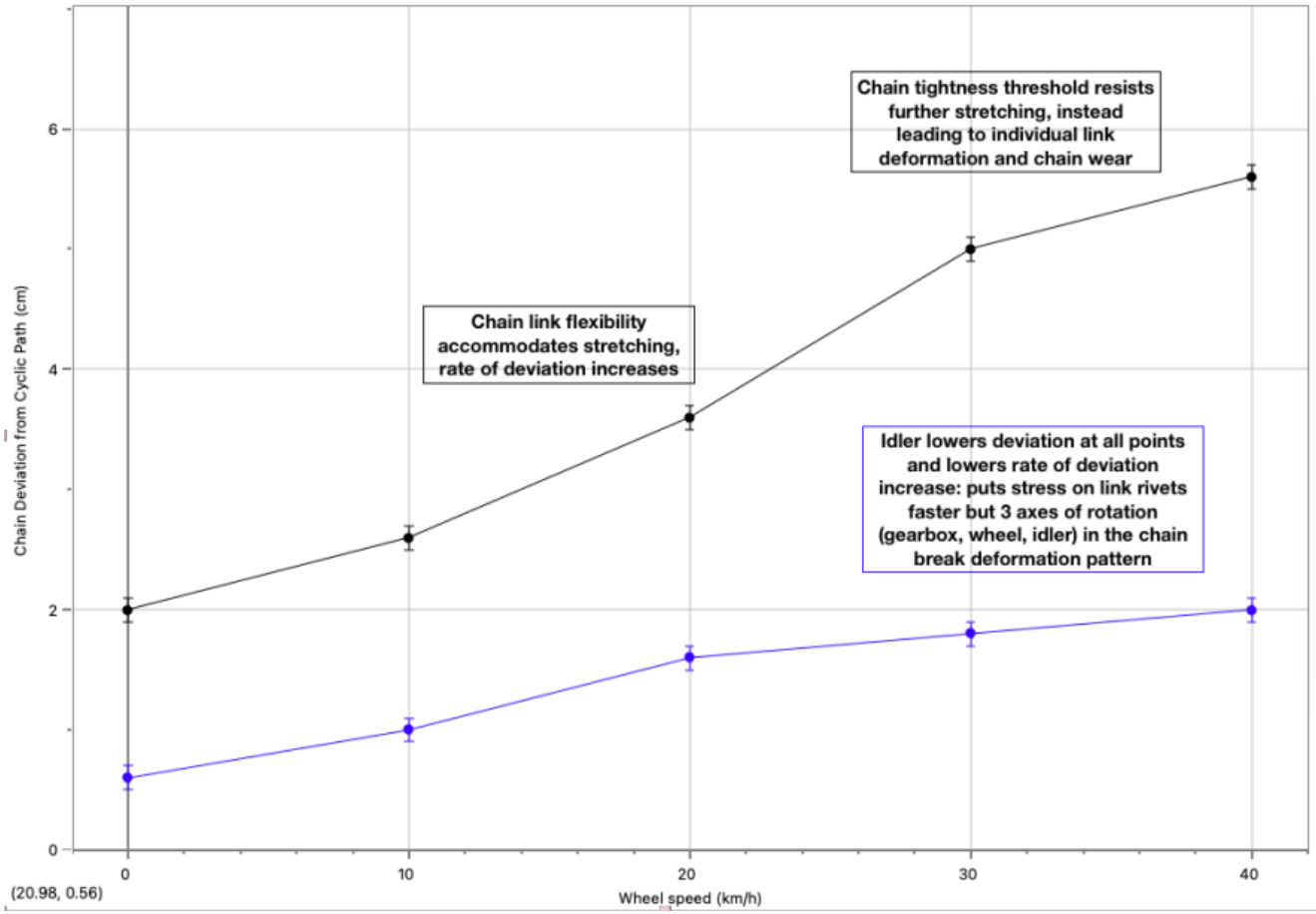


Figure: Figure 13: Chain deviation before/after idler install.

Initial Chain Failure

Chain derailed above 20 km/h prior to idler fix.

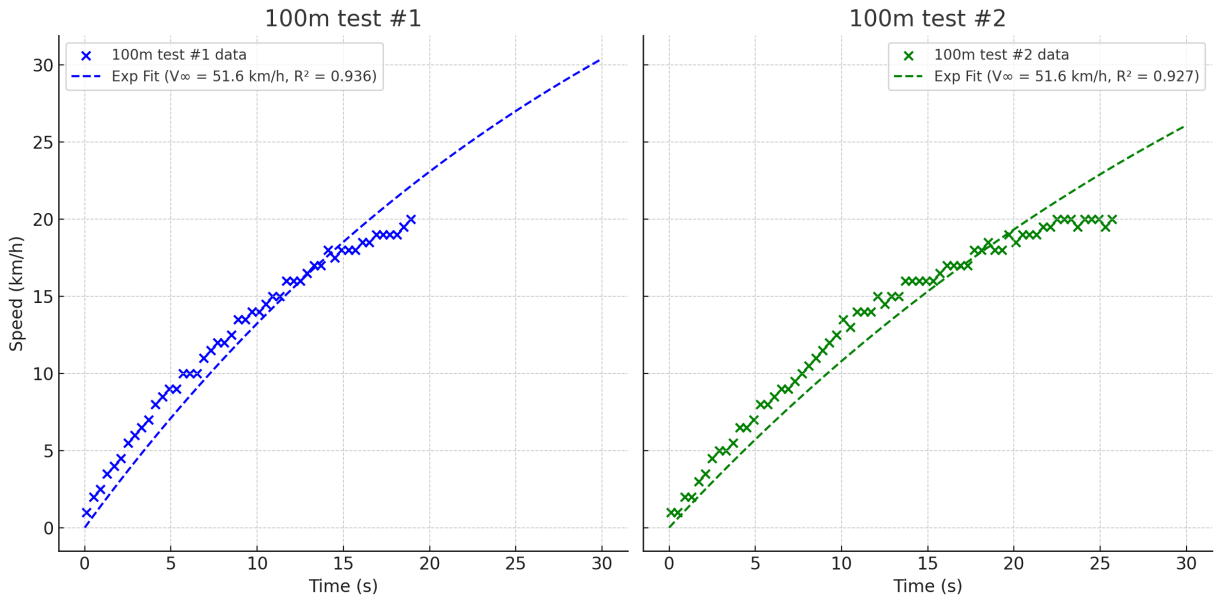


Figure: Figure 14: Pre-fix chain derailment testing.

Acceleration After Fix

Post-fix acceleration stabilized drivetrain operation.

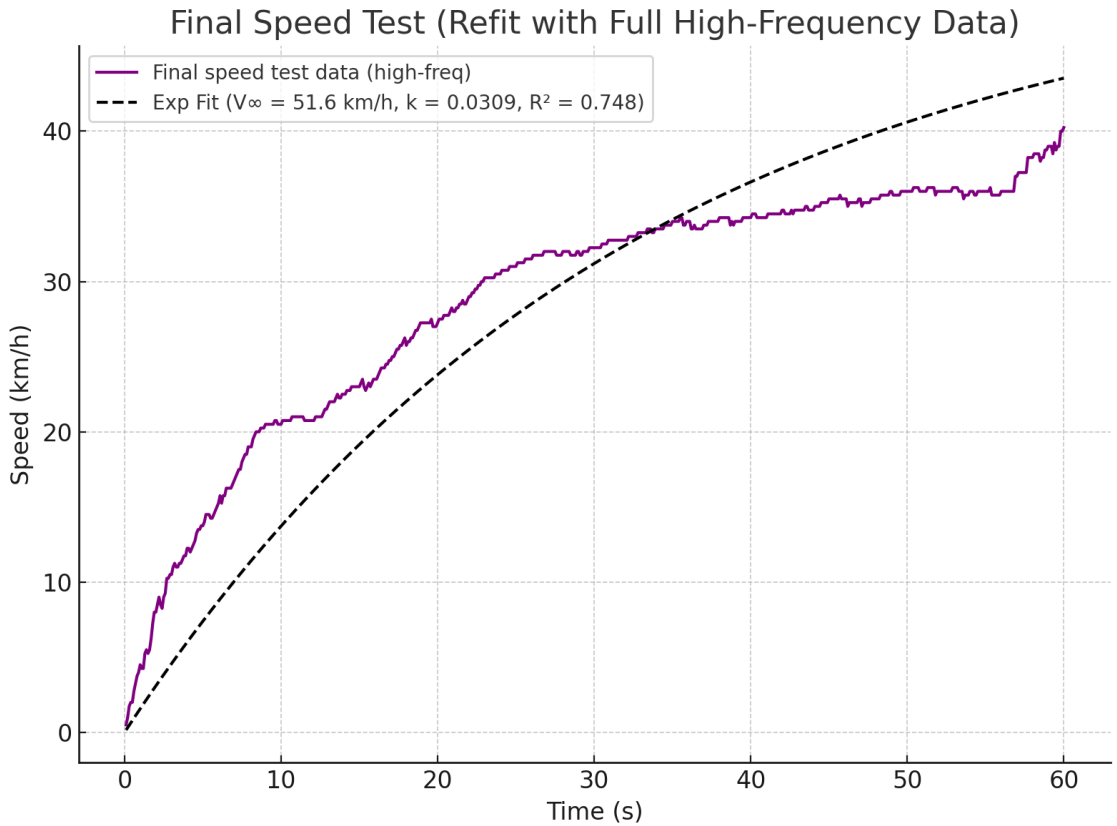


Figure: Figure 15: Final 0–40 km/h acceleration.

Powertrain Architecture

Boosted 12V-24V powertrain maximized efficiency and runtime.

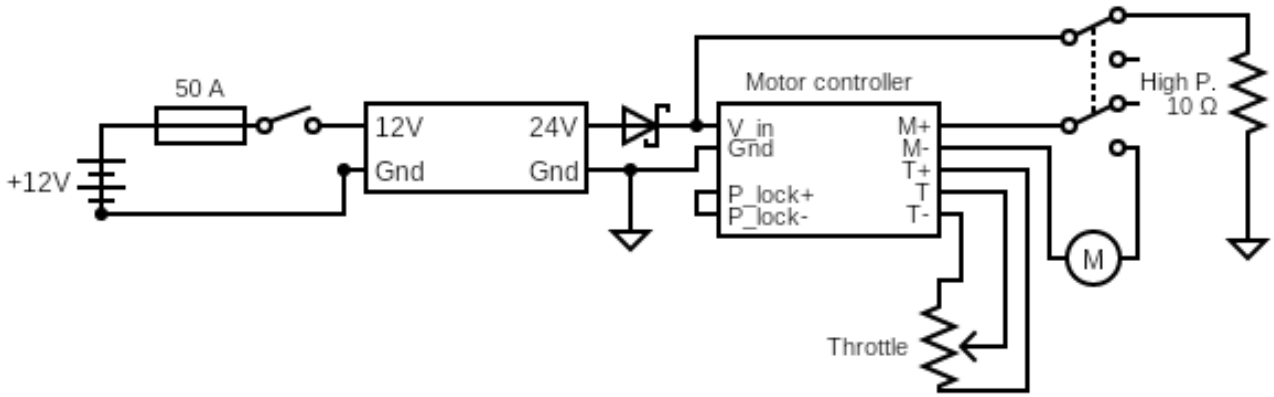


Figure: Figure 16: Full electrical diagram, before self-driving.

Self-Driving Overview

- Fully custom vision-based steering control.
- Combined mechanical + electrical + ML pipelines.
- Three front-mounted 1440p cameras for visual input.
- Control loop: Perception $\rightarrow$ Inference $\rightarrow$ Actuation.

Mechanical Integration

- Motorized steering via gear-driven actuator mounted to steering column.
- Emergency override kill switch integrated into main power circuit.
- Arduino Nano provided low-level interface between motor controller and main compute.

Figure: Figure 17: Motorized steering assembly.

Hardware Data Flow

- 3x 1440p cameras mounted on front hood.
- USB 3.0 capture to local compute node.
- Inference computed onboard via CPU/GPU.
- Arduino Nano converts predicted angles to PWM control signals.

Machine Learning Pipeline

- Dataset generation: manual driving recorded with paired images + steering angles.
- Supervised training on full dataset.
- Final model: regression network predicting continuous steering angle.
- **Software stack:** OpenCV, TensorFlow/Keras, Python.

Computer Vision Preprocessing

- Camera frames filtered via OpenCV:
  - Edge detection (Canny filter).
  - Lane extraction via HoughLines.
  - Perspective transforms for road geometry.
- Output: simplified feature maps for neural net input.

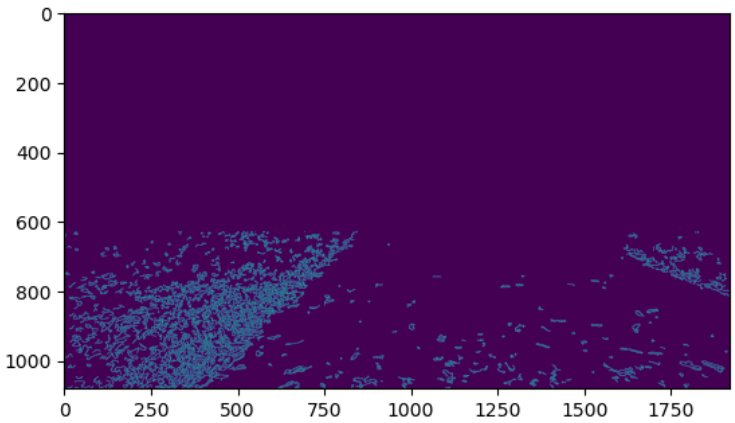


Figure: Figure 19: Sample edge map from preprocessing.

Neural Network Architecture

- Modified convolutional behavioral cloning model:
- Layers:
  - Convolution + ReLU + MaxPool $\times$ 3
  - Fully Connected layers
  - Single output node: predicted steering angle.
- Similar to NVIDIA End-to-End model.

$$\theta = f(\text{image features})$$

MNIST / Baseline Models

- Initial training conducted on MNIST to validate pipeline.
- Confirmed proper data flow, loss function convergence.
- MNIST $\rightarrow$ Behavioral cloning transfer learning.
- Key point: validated model could generalize on real-world driving frames.

Behavioral Cloning in Application

- Trained network successfully generalized to varied track geometries.
- Smooth steering response under both turns and straights.
- Inference latency: $<$50 ms (mechanical latency is much higher).

Video Reference: *Self-Driving Turn Clip.*

Sample Control Output

- Predicted heading angle converted to PWM via Arduino.
- Dead-zone filtering to stabilize minor prediction noise.
- Safety bound clamping limits max steering deflection.

$$PWM = a \cdot \theta + b$$

PID Control Mapping

- Neural network predicts θ_{desired}.
- PID maps angle error into PWM signal:

$$e(t) = \theta_{\text{desired}}(t) - \theta_{\text{current}}(t)$$

$$\text{PWM}(t) = K_P e(t) + K_I \int_0^t e(\tau)d\tau + K_D \frac{de(t)}{dt} + b$$

- Baseline offset b centers steering PWM range.
- PID terms smooth and stabilize rapid steering corrections.

Real-World Performance

- Stable control maintained through a straight trac.
- Self-driving completed both straight and curved segments.
- Full kill-switch handover in case of system faults.

Video Reference: *Self-Driving Straight Clip.*

Thank You!